B-Trees

Motivation for B-Trees

- Index structures for large datasets cannot be stored in main memory
- Storing it on disk requires different approach to efficiency

- Assuming that a disk spins at 3600 RPM, one revolution occurs in 1/60 of a second, or 16.7ms
- Crudely speaking, one disk access takes about the same time as 200,000 instructions

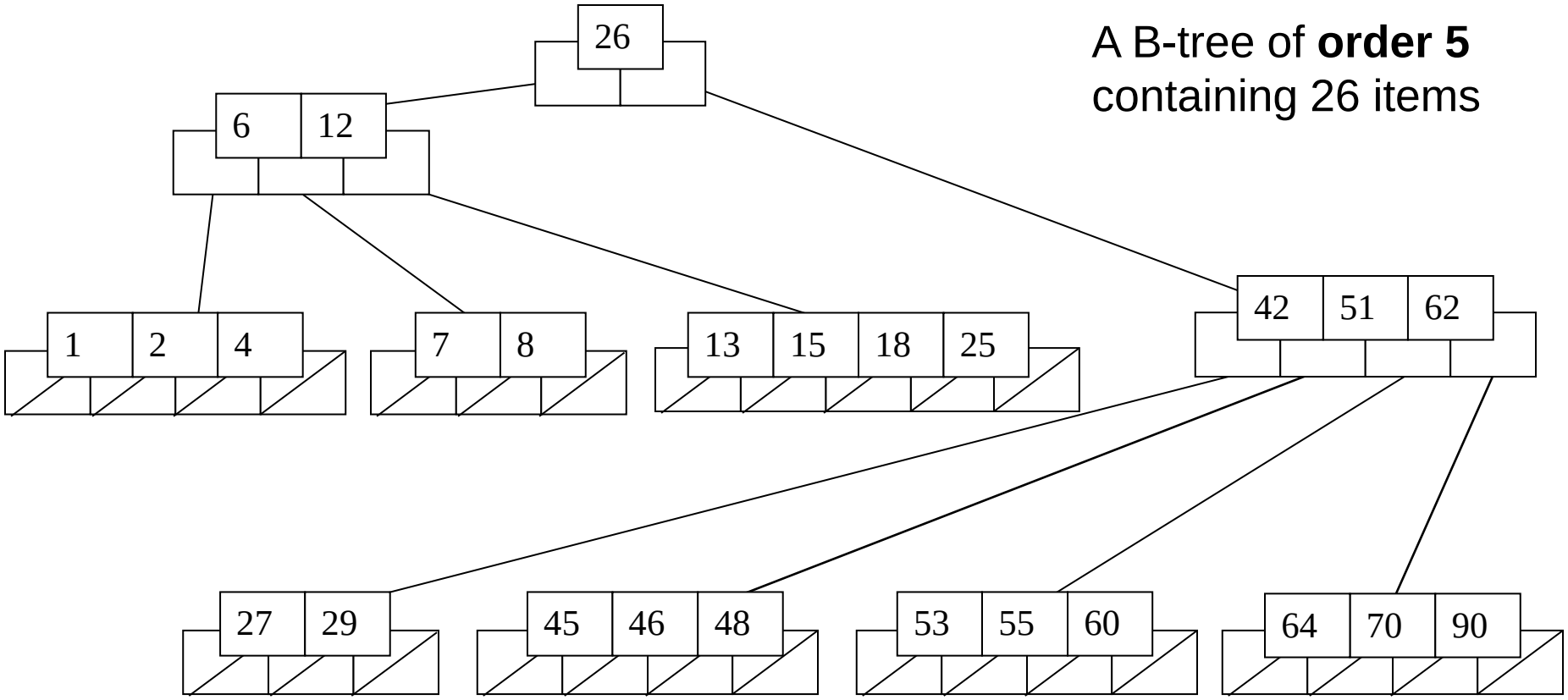
Motivation (cont.)

- Assume that we use an AVL tree to store about 20 million records
- We end up with a **very** deep binary tree with lots of different disk accesses; $\log_2 20{,}000{,}000$ is about 24, so this takes about 0.2 seconds
- We know we can't improve on the log *n* lower bound on search for a binary tree
- But, the solution is to use more branches and thus reduce the height of the tree!
  - As branching increases, depth decreases

Definition of a B-tree

- A B-tree of order m is an m-way tree (i.e., a tree where each node may have up to m children) in which:
  1. the number of keys in each non-leaf node is one less than the number of its children and these keys partition the keys in the children in the fashion of a search tree
  2. all leaves are on the same level
  3. all non-leaf nodes except the root have at least $\lceil m / 2 \rceil$ children
  4. the root is either a leaf node, or it has from two to m children
  5. a leaf node contains no more than $m - 1$ keys
- The number m should always be odd

An example B-Tree



A B-tree of **order 5** containing 26 items

Note that all the leaves are at the same level

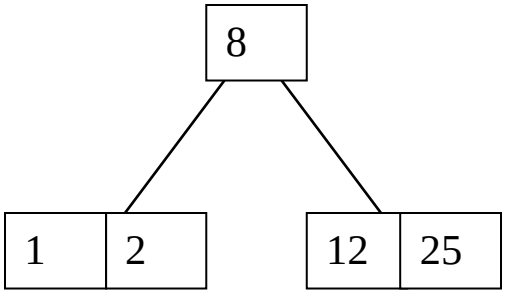
Constructing a B-tree

- Suppose we start with an empty B-tree and keys arrive in the following order: 1 12 8 2 25 6 14 28 17 7 52 16 48 68 3 26 29 53 55 45
- We want to construct a B-tree of order 5
- The first four items go into the root:

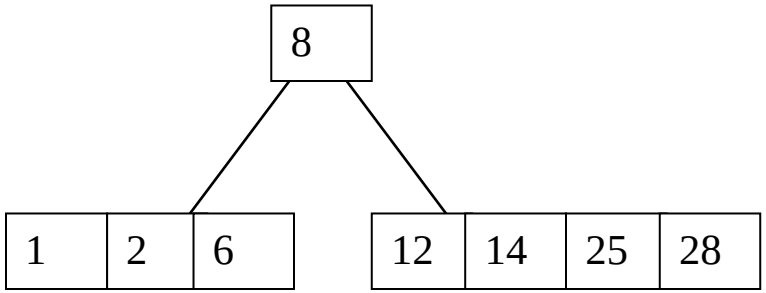
1	2	8	12

- To put the fifth item in the root would violate condition 5
- Therefore, when 25 arrives, pick the middle key to make a new root

1 12 8 2 25 6 14 28 17 7 52 16 48 68 3 26 29 53 55 45

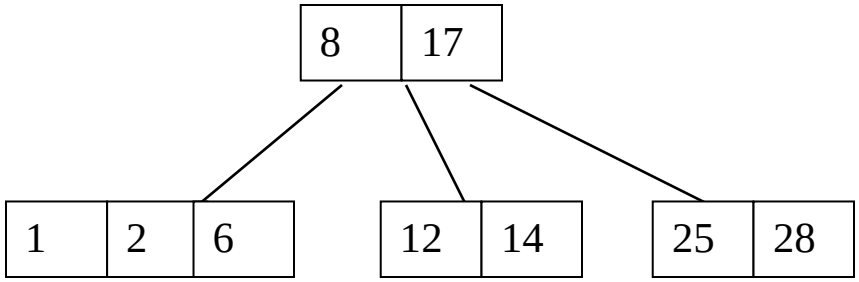


6, 14, 28 get added to the leaf nodes:

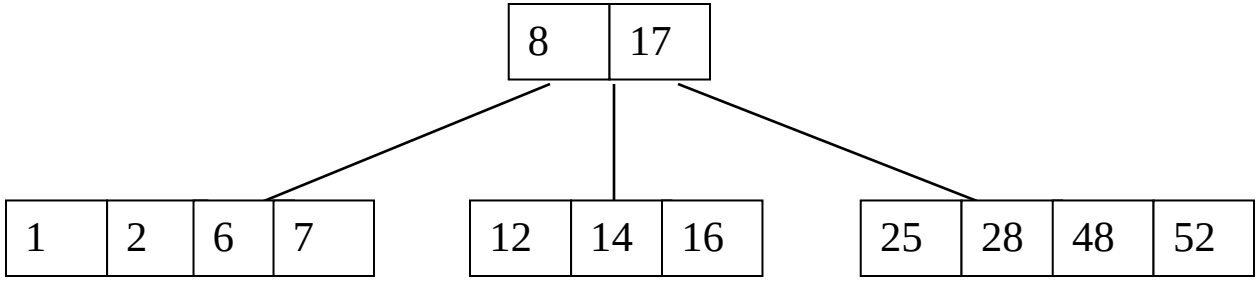


1 12 8 2 25 6 14 28 17 7 52 16 48 68 3 26 29 53 55 45

Adding 17 to the right leaf node would over-fill it, so we take the middle key, promote it (to the root) and split the leaf

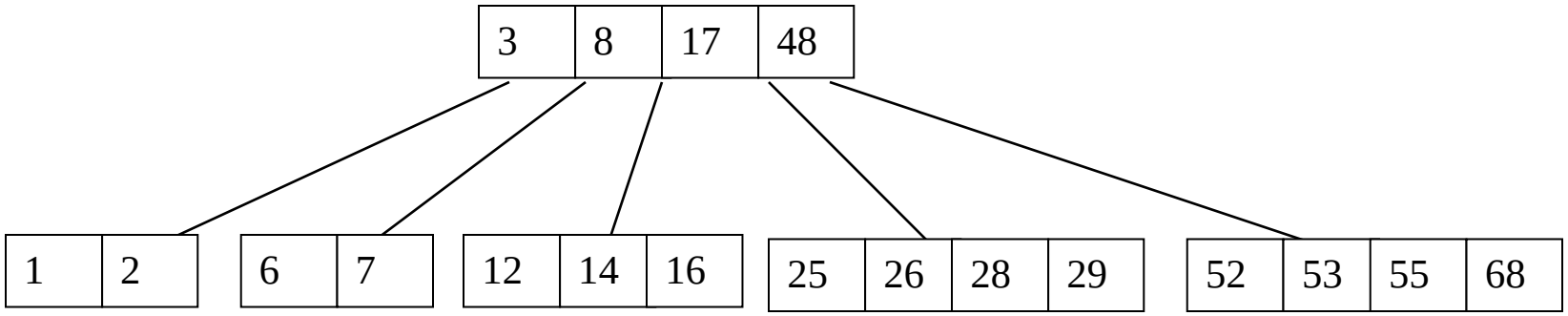


7, 52, 16, 48 get added to the leaf nodes



1 12 8 2 25 6 14 28 17 7 52 16 48 68 3 26 29 53 55 45

Adding 68 causes us to split the right most leaf, promoting 48 to the root, and adding 3 causes us to split the left most leaf, promoting 3 to the root; 26, 29, 53, 55 then go into the leaves

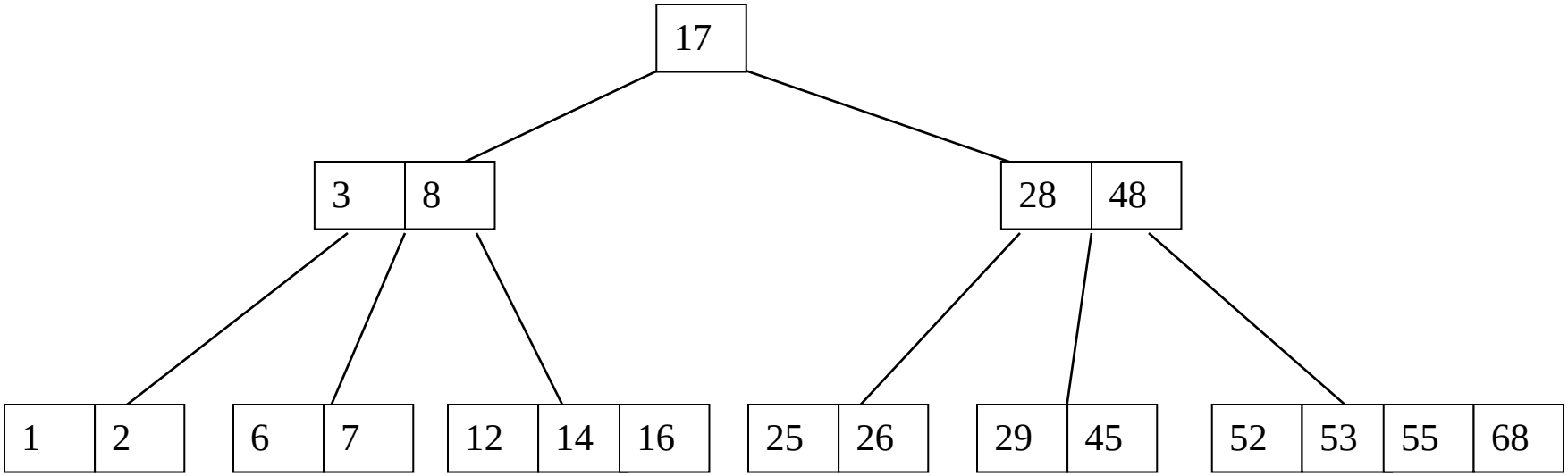


Adding 45 causes a split of

25	26	28	29

and promoting 28 to the root then causes the root to split

Constructing a B-tree (contd.)



Inserting into a B-Tree

- Attempt to insert the new key into a leaf
- If this would result in that leaf becoming too big, split the leaf into two, promoting the middle key to the leaf's parent
- If this would result in the parent becoming too big, split the parent into two, promoting the middle key
- This strategy might have to be repeated all the way to the top
- If necessary, the root is split in two and the middle key is promoted to a new root, making the tree one level higher

Exercise in Inserting a B-Tree

- Insert the following keys to a 5-way B-tree:
- 3, 7, 9, 23, 45, 1, 5, 14, 25, 24, 13, 11, 8, 19, 4, 31, 35, 56
-

Removal from a B-tree

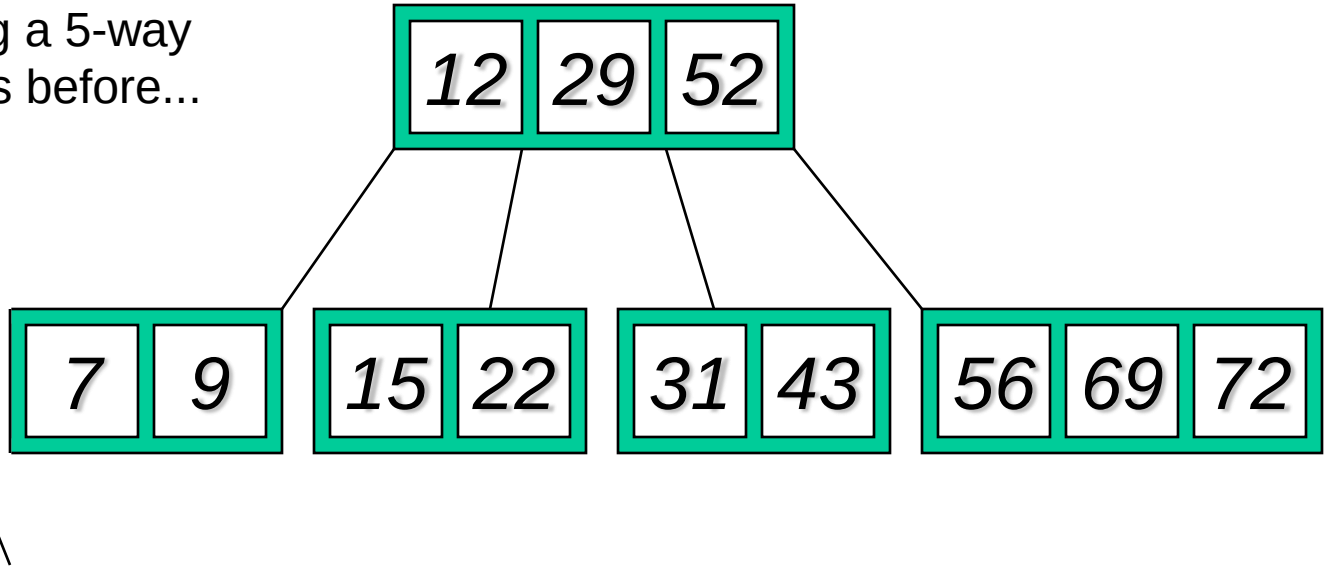
- During insertion, the key always goes *into* a *leaf*. For deletion we wish to remove *from* a leaf. There are three possible ways we can do this:
- 1 - If the key is already in a leaf node, and removing it doesn't cause that leaf node to have too few keys, then simply remove the key to be deleted.
- 2 - If the key is *not* in a leaf then it is guaranteed (by the nature of a B-tree) that its predecessor or successor will be in a leaf -- in this case we can delete the key and promote the predecessor or successor key to the non-leaf deleted key's position.

Removal from a B-tree (2)

- If (1) or (2) lead to a leaf node containing less than the minimum number of keys then we have to look at the siblings immediately adjacent to the leaf in question:
  - 3: if one of them has more than the min. number of keys then we can promote one of its keys to the parent and take the parent key into our lacking leaf
  - 4: if neither of them has more than the min. number of keys then the lacking leaf and one of its neighbours can be combined with their shared parent (the opposite of promoting a key) and the new leaf will have the correct number of keys; if this step leave the parent with too few keys then we repeat the process up to the root itself, if required

Type #1: Simple leaf deletion

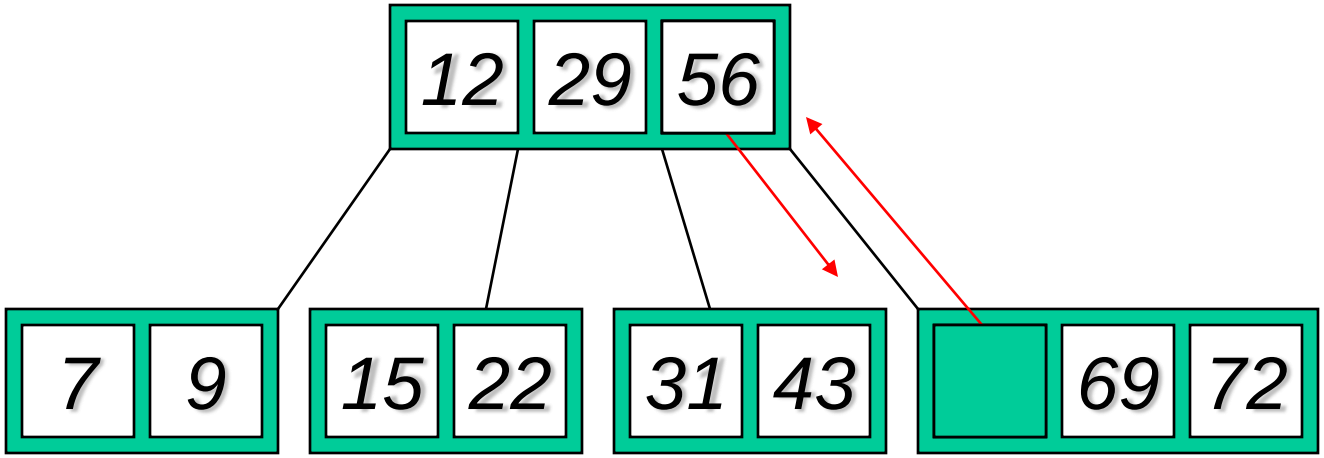
Assuming a 5-way B-Tree, as before...



Delete 2: Since there are enough keys in the node, just delete it

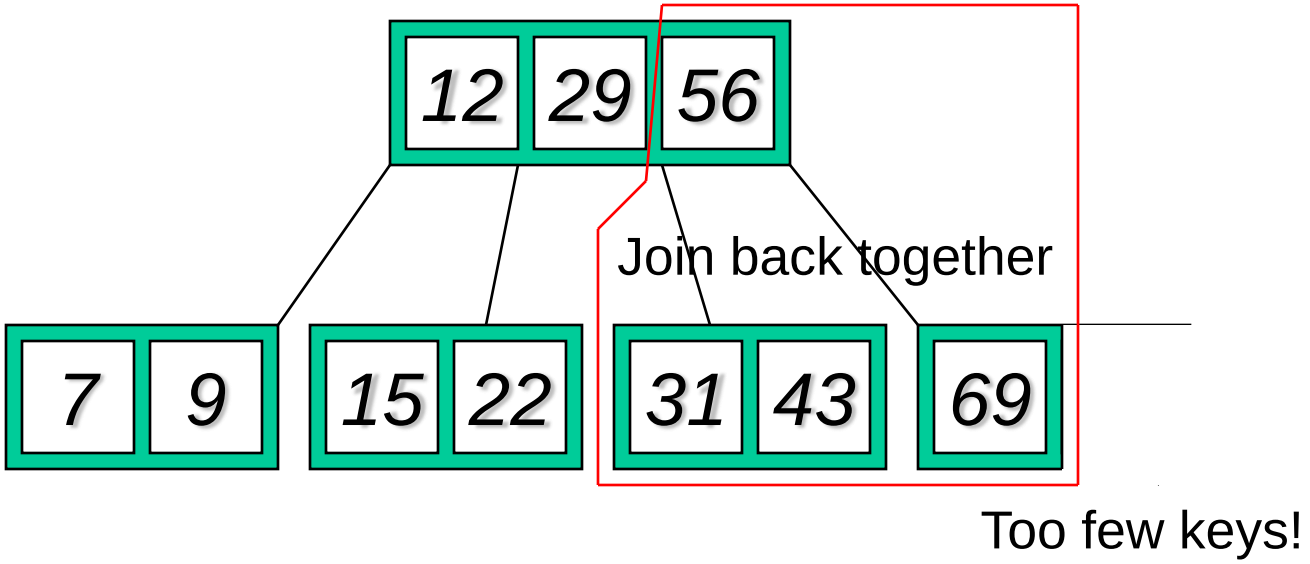
Note when printed: this slide is animated

Type #2: Simple non-leaf deletion



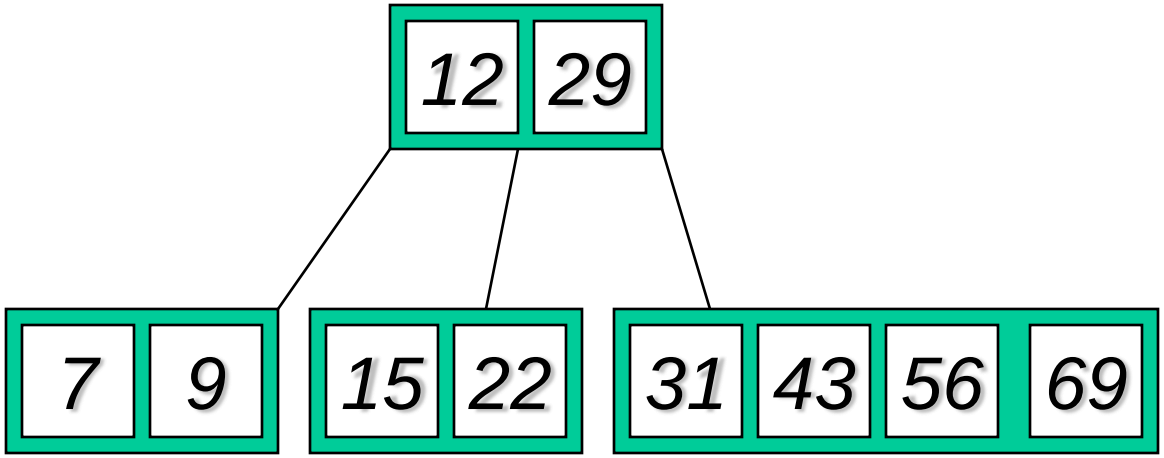
Note when printed: this slide is animated

Type #4: Too few keys in node and its siblings



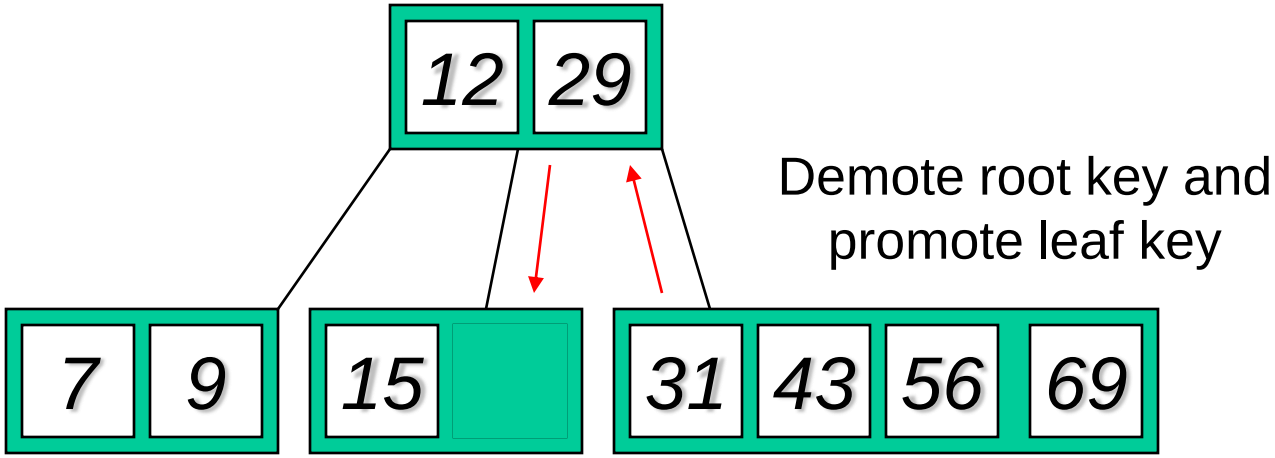
Note when printed: this slide is animated

Type #4: Too few keys in node and its siblings



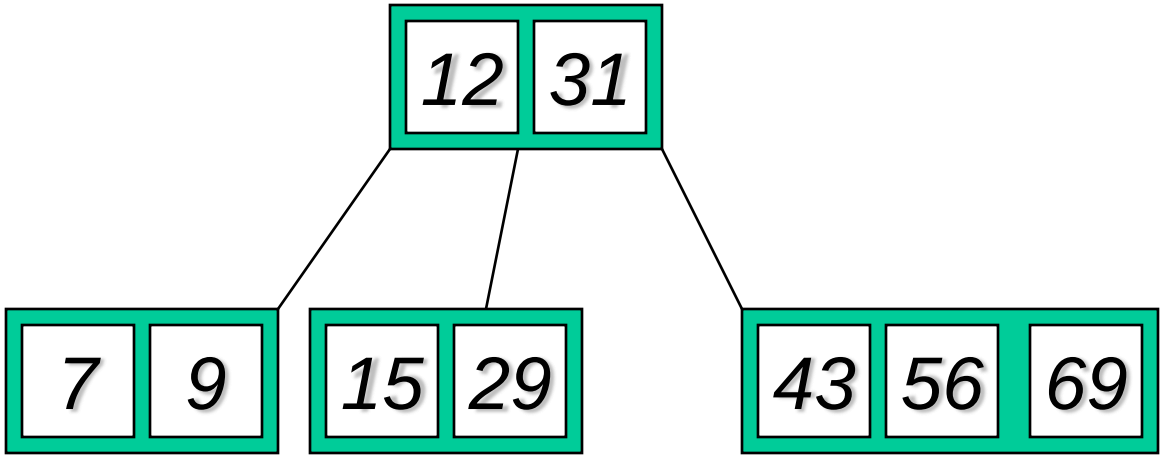
Note when printed: this slide is animated

Type #3: Enough siblings



Note when printed: this slide is animated

Type #3: Enough siblings



Note when printed: this slide is animated

```
struct btreeNode {
        int val[MAX + 1], count;
        struct btreeNode *link[MAX + 1];
  };
```

```
struct btreeNode * createNode(int val, struct btreeNode *child) {
        struct btreeNode *newNode;
        newNode = (struct btreeNode *)malloc(sizeof(struct
btreeNode));
        newNode->val[1] = val;
        newNode->count = 1;
        newNode->link[0] = root;
        newNode->link[1] = child;
        return newNode;
 }
```

```
/* Places the value in appropriate position */
  void addValToNode(int val, int pos, struct
btreeNode *node,
                    struct btreeNode *child) {
        int j = node->count;
        while (j > pos) {
                node->val[j + 1] = node->val[j];
                node->link[j + 1] = node->link[j];
                j--;
        }
        node->val[j + 1] = val;
        node->link[j + 1] = child;
        node->count++;
  }
```

```
void splitNode (int val, int *pval, int pos, struct btreeNode
*node,
    struct btreeNode *child, struct btreeNode **newNode) {
      int median, j;

      if (pos > MIN)
            median = MIN + 1;
      else
            median = MIN;

      *newNode = (struct btreeNode *)malloc(sizeof(struct
btreeNode));
      j = median + 1;
      while (j <= MAX) {
            (*newNode)->val[j - median] = node->val[j];
            (*newNode)->link[j - median] = node->link[j];
            j++;
}
```

```
node->count = median;
        (*newNode)->count = MAX - median;

        if (pos <= MIN) {
                addValToNode(val, pos, node, child);
        } else {
                addValToNode(val, pos - median,
*newNode, child);
        }
        *pval = node->val[node->count];
        (*newNode)->link[0] = node->link[node->count];
        node->count--;
  }
```

Exercise in Removal from a B-Tree

- Given 5-way B-tree created by these data (last exercise):
- 3, 7, 9, 23, 45, 1, 5, 14, 25, 24, 13, 11, 8, 19, 4, 31, 35, 56
- Add these further keys: 2, 6,12
- Delete these keys: 4, 5, 7, 3, 14
-

Analysis of B-Trees

- The maximum number of items in a B-tree of order m and height h:

root	$m - 1$
level 1	$m(m - 1)$
level 2	$m^2(m - 1)$
. . .	
level h	$m^h(m - 1)$

- So, the total number of items is

$$(1 + m + m^2 + m^3 + \ldots + m^h)(m - 1) =$$

$$[(m^{h+1} - 1)/ (m - 1)] (m - 1) = \boldsymbol{m^{h+1} - 1}$$

- When $m = 5$ and $h = 2$ this gives $5^3 - 1 = 124$

Reasons for using B-Trees

- When searching tables held on disc, the cost of each disc transfer is high but doesn't depend much on the amount of data transferred, especially if consecutive items are transferred
  - If we use a B-tree of order 101, say, we can transfer each node in one disc read operation
  - A B-tree of order 101 and height 3 can hold $101^4 - 1$ items (approximately 100 million) and any item can be accessed with 3 disc reads (assuming we hold the root in memory)
- If we take $m = 3$, we get a **2-3 tree**, in which non-leaf nodes have two or three children (i.e., one or two keys)
  - B-Trees are always balanced (since the leaves are all at the same level), so 2-3 trees make a good type of balanced tree

Comparing Trees

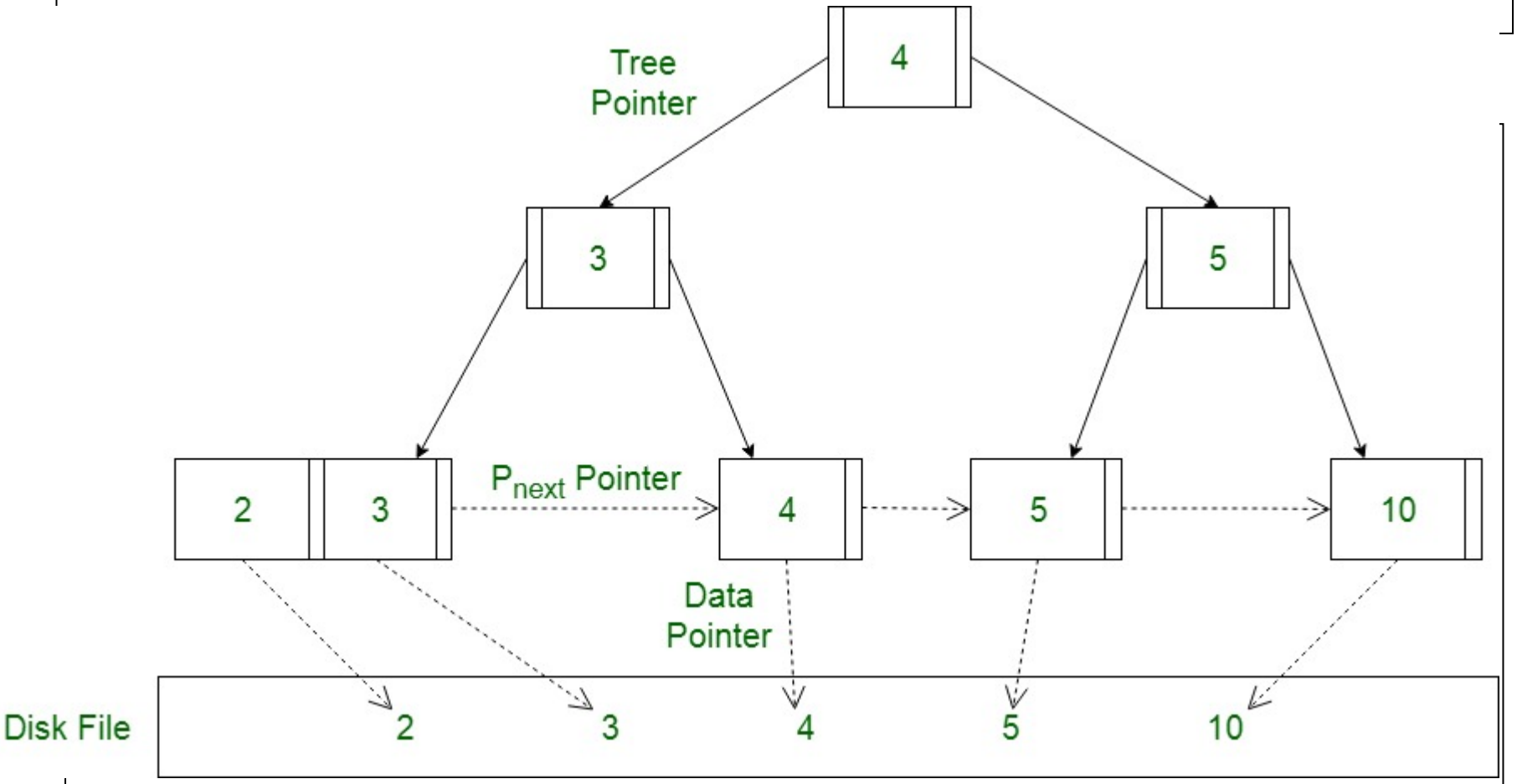
- Binary trees
  - Can become *unbalanced* and *lose* their good time complexity (big O)
  - AVL trees are strict binary trees that *overcome the balance problem*
  - Heaps remain balanced but only *prioritise* (not order) the keys

- Multi-way trees
  - B-Trees can be *m*-way, they can have any (odd) number of children
  - One B-Tree, the 2-3 (or 3-way) B-Tree, *approximates* a permanently balanced binary tree, exchanging the AVL tree's balancing operations for insertion and (more complex) deletion operations

S.NO	B TREE	B+ TREE
1.	All internal and leaf nodes have data pointers	Only leaf nodes have data pointers
2.	Since all keys are not available at leaf, search often takes more time.	All keys are at leaf nodes, hence search is faster and accurate..
3.	No duplicate of keys is maintained in the tree.	Duplicate of keys are maintained and all nodes are present at leaf.
4.	Insertion takes more time and it is not predictable sometimes.	Insertion is easier and the results are always the same.
5.	Deletion of internal node is very complex and tree has to undergo lot of transformations.	Deletion of any node is easy because all node are found at leaf.
6.	Leaf nodes are not stored as structural linked list.	Leaf nodes are stored as structural linked list.
7.	No redundant search keys are present..	Redundant search keys may be present..

B+ Tree

- B+ tree eliminates the drawback B-tree used for indexing
- It store data pointers only at the leaf nodes of the tree.
- Data pointers are present only at the leaf nodes, the leaf nodes must necessarily store all the key values along with their corresponding data pointers to the disk file block, in order to access them.
- Leaf nodes are linked to providing ordered access to the records.
- The leaf nodes, therefore form the first level of the index, with the internal nodes forming the other levels of a multilevel index.
- Some of the key values of the leaf nodes also appear in the internal nodes, to simply act as a medium to control the searching of a record.


Tree
Pointer
4
3
5
2
3
P_{next} Pointer
4
5
10
Data
Pointer
Disk File
2
3
4
5
10

Insertion

- Each node except root can have a maximum of **M** children and at least **ceil(M/2)** children.
- Each node can contain a maximum of **M – 1** keys and a minimum of **ceil(M/2) – 1** keys.
- The root has at least two children and atleast one search key.
- While insertion overflow of the node occurs when it contains more than **M – 1** search key values.

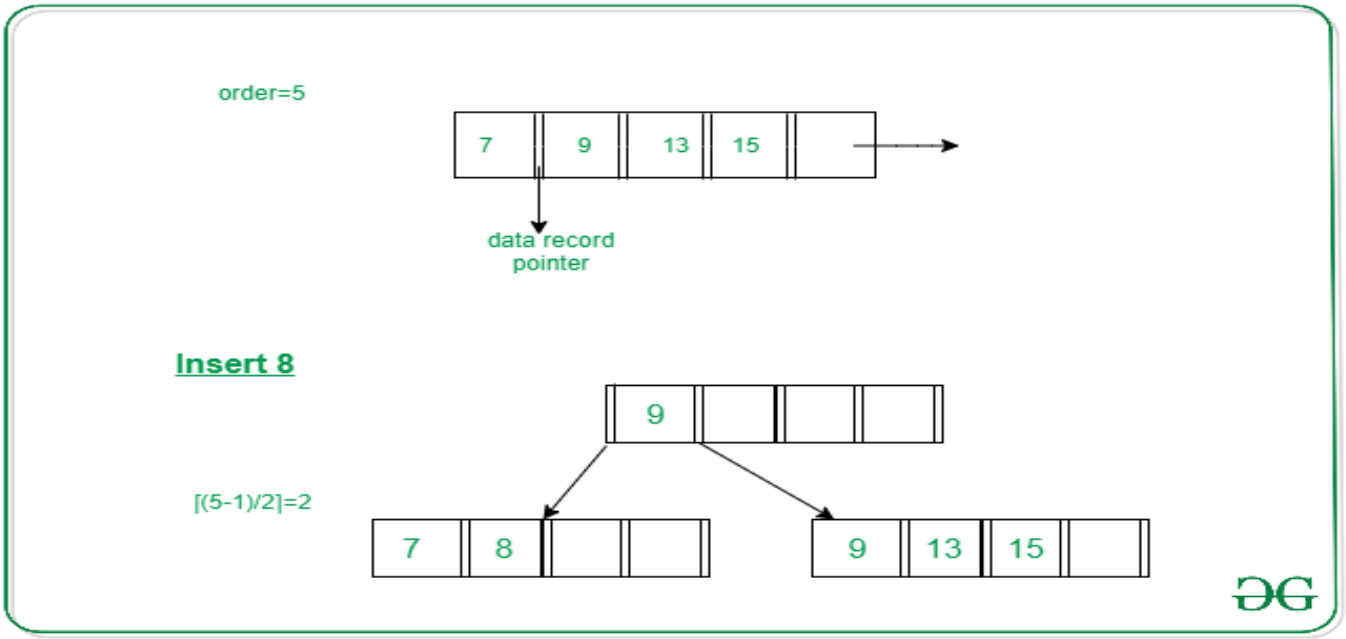
Here **M** is the order of B+ tree.

Steps for insertion in B+ Tree

- Every element is inserted into a leaf node. So, go to the appropriate leaf node.
- Insert the key into the leaf node in increasing order only if there is no overflow. If there is an overflow go ahead with the following steps mentioned below to deal with overflow while maintaining the B+ Tree properties.

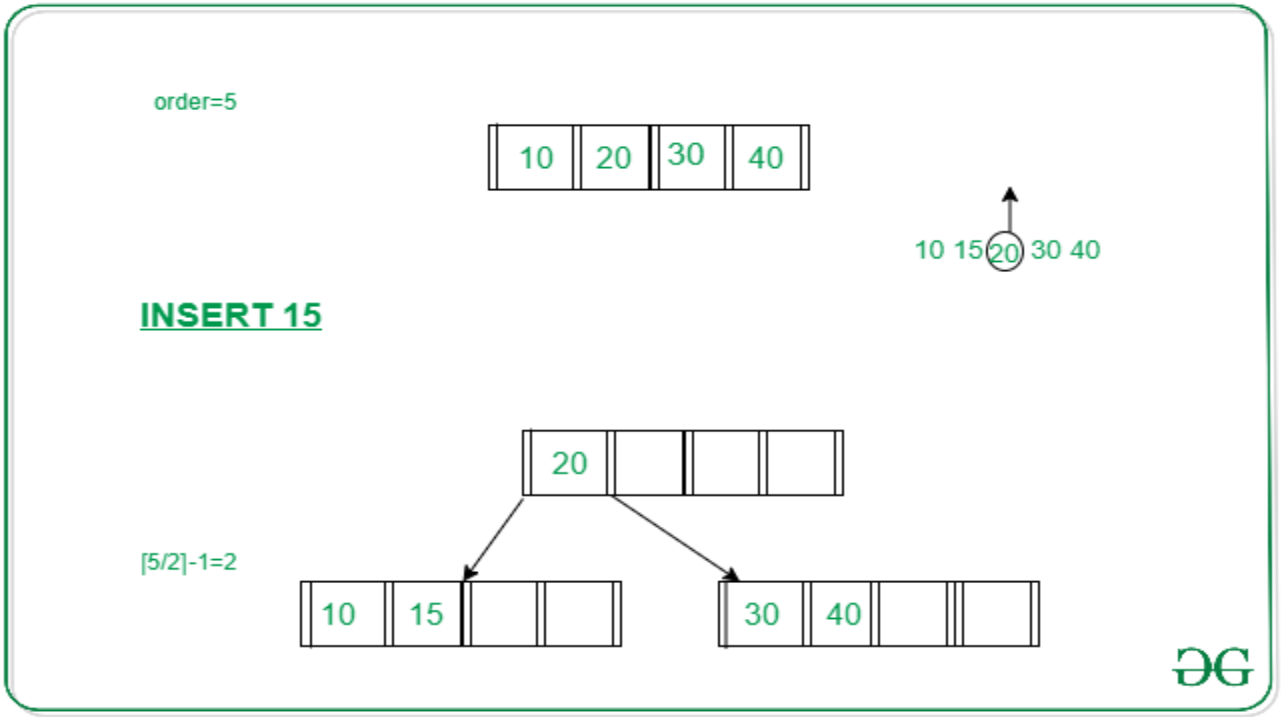
Case 1: Overflow in leaf node

- Split the leaf node into two nodes.
- First node contains **ceil((m-1)/2)** values.
- Second node contains the remaining values.
- Copy the smallest search key value from second node to the parent node.(Right biased)



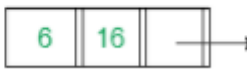
Case 2: Overflow in non-leaf node

- Split the non leaf node into two nodes.
- First node contains ceil(m/2)-1 values.
- Move the smallest among remaining to the parent.
- Second node contains the remaining keys.



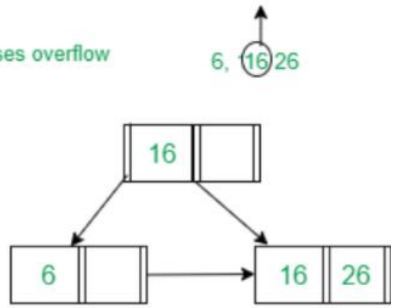
Insert the following key values 6, 16, 26, 36, 46 on a B+ tree with order = 3.

Insert 6, 16

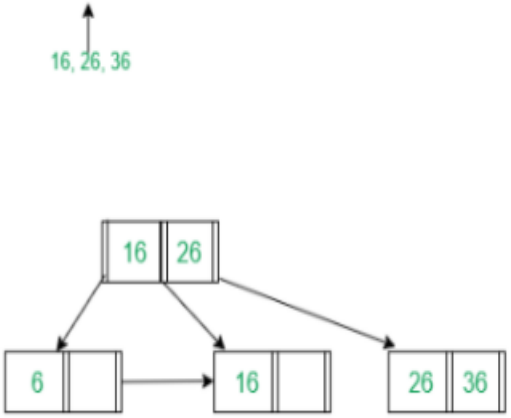


Insert 26

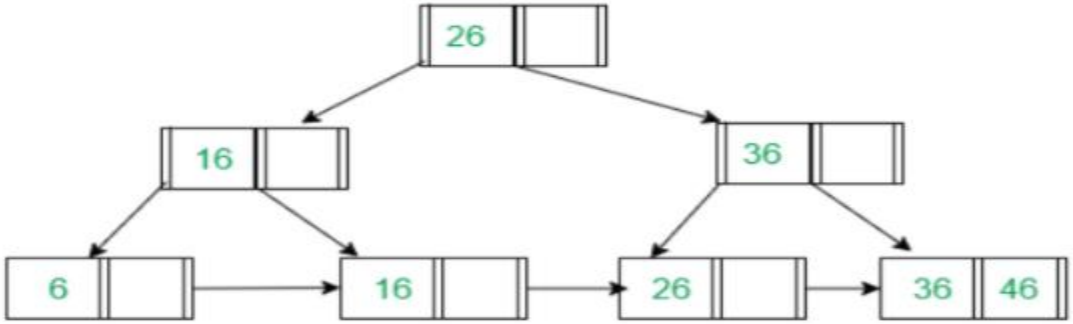
causes overflow



Insert 36

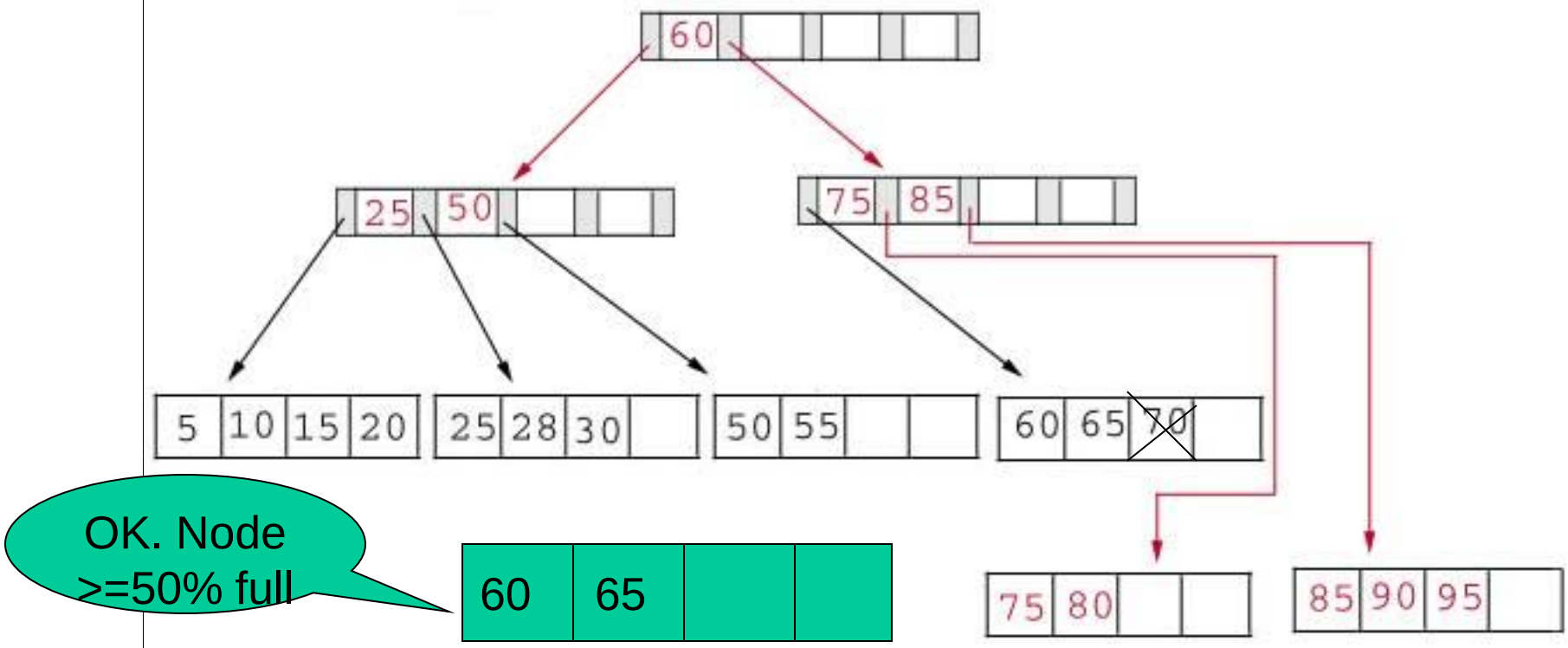


Insert 46



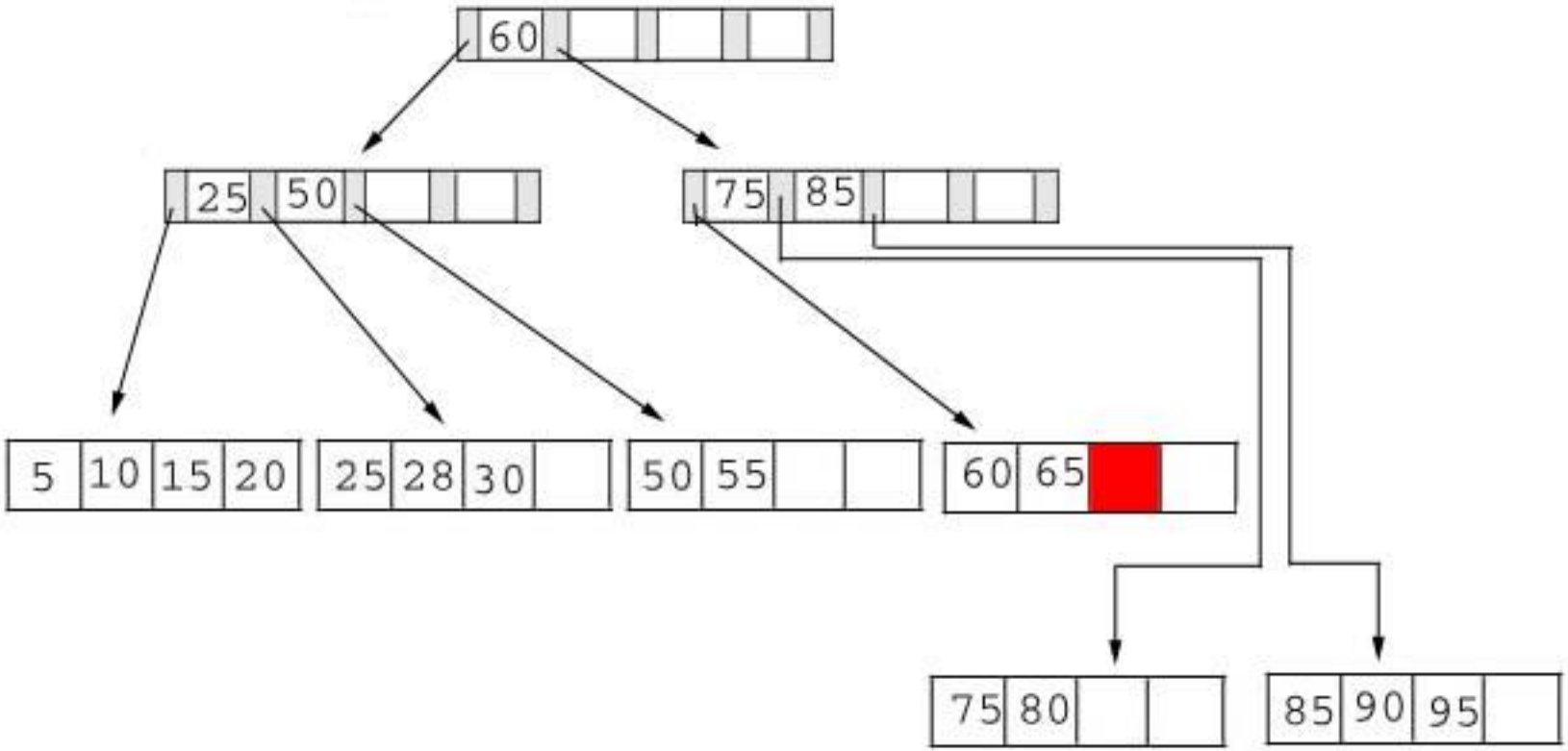
Deletion

- Same as insertion, the tree has to be rebuild if the deletion result violate the rule of B+ tree.
- Example #1: delete 70 from the tree



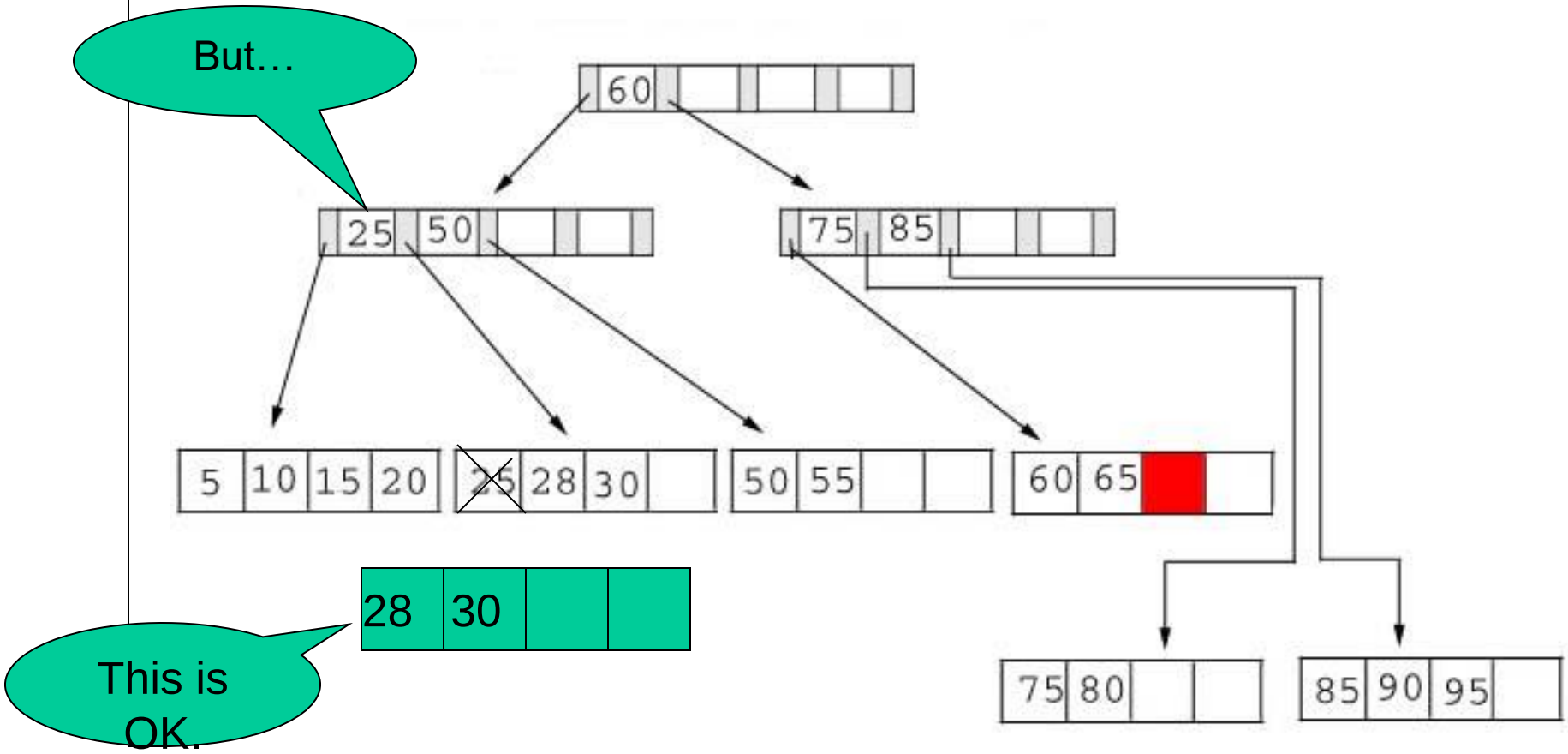
Deletion

- Result:



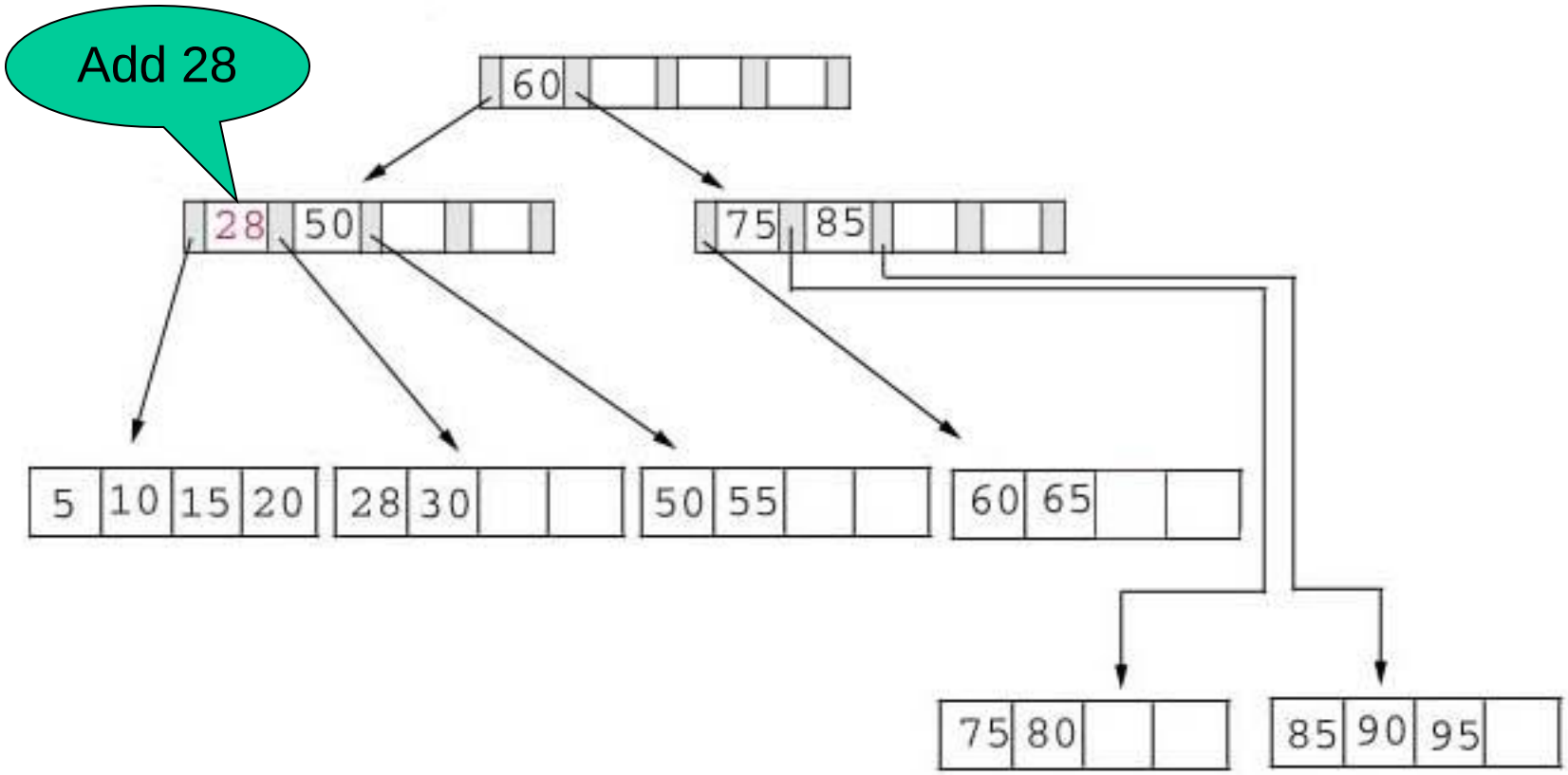
Deletion

Example #2: delete 25 from below tree, but 25 appears in the index page.



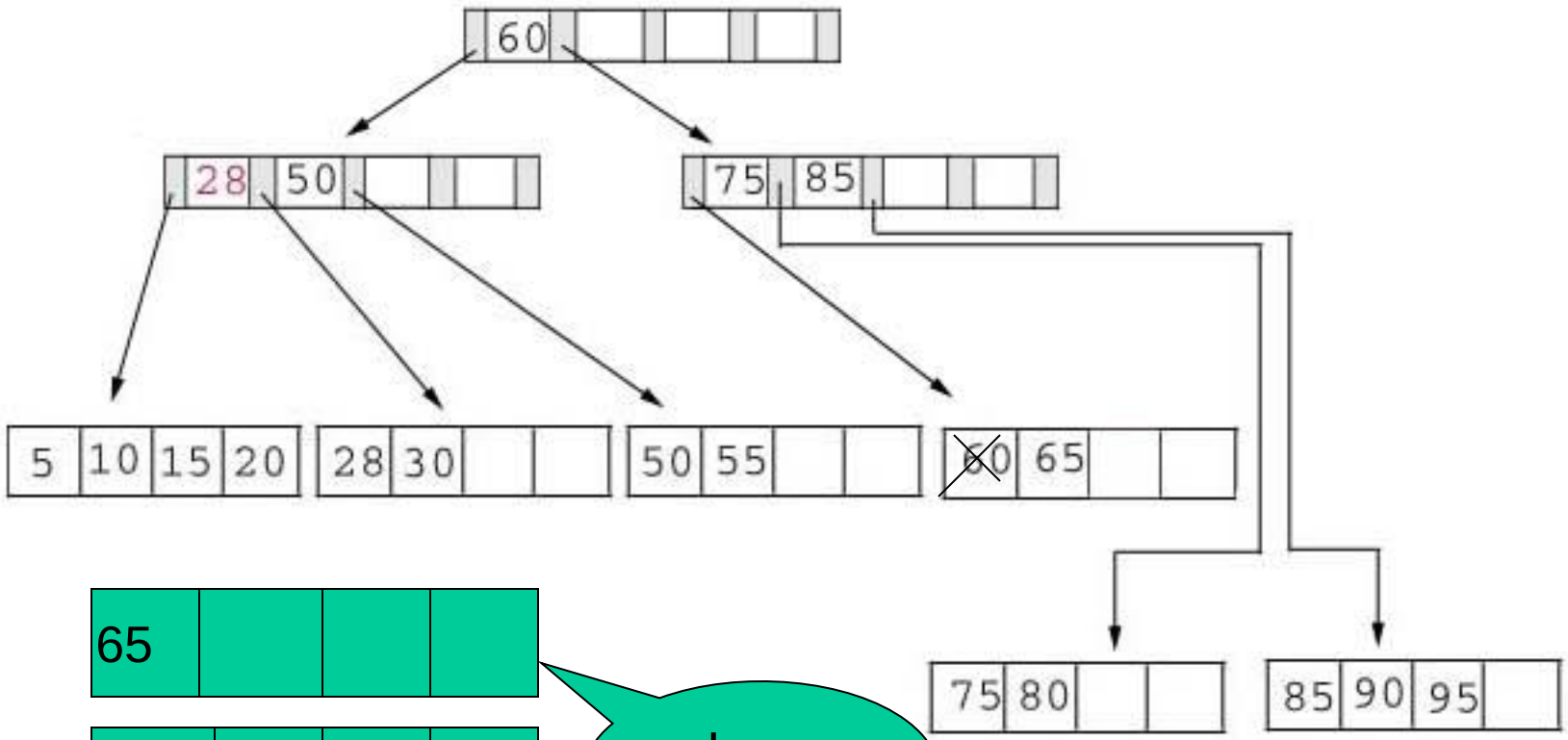
Deletion

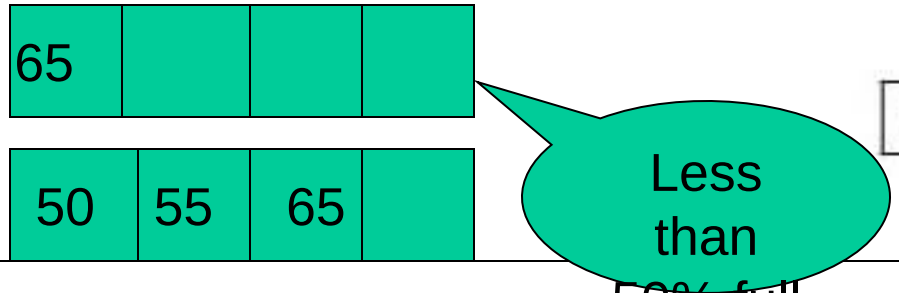
- Result: replace 28 in the index page.



Deletion

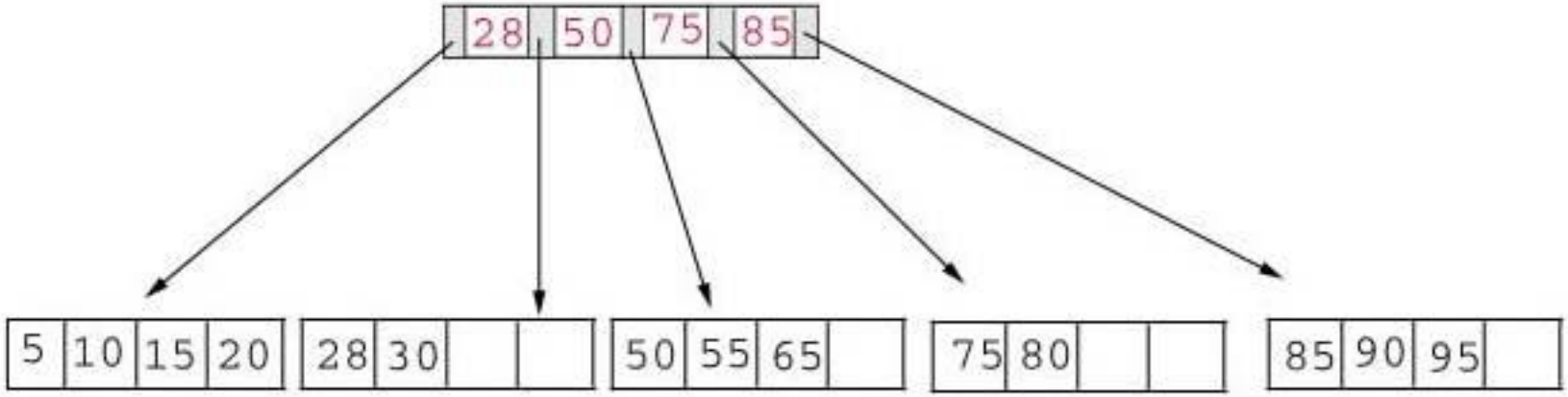
- Example #3: delete 60 from the below tree





Deletion

- Result: delete 60 from the index page and combine the rest of index pages.



Deletion

- Delete algorithm for B+ trees

Data Page Below Fill Factor	Index Page Below Fill Factor	Action
NO	NO	Delete the record from the leaf page. Arrange keys in ascending order to fill void. If the key of the deleted record appears in the index page, use the next key to replace it.
YES	NO	Combine the leaf page and its sibling. Change the index page to reflect the change.
YES	YES	1. Combine the leaf page and its sibling. 2. Adjust the index page to reflect the change. 3. Combine the index page with its sibling. Continue combining index pages until you reach a page with the correct fill factor or you reach the root page.

Conclusion

- For a B+ Tree:
- It is “easy” to maintain its balance
  - Insert/Deletion complexity $O(\log_{M/2})$
- The searching time is shorter than most of other types of trees because branching factor is high

B+Trees and DBMS

- Used to index primary keys
- Can access records in $O(\log_{M/2})$ traversals (height of the tree)
- Interior nodes contain Keys only
  - Set node sizes so that the M-1 keys and M pointers fits inside a single block on disk
    - E.g., block size 4096B, keys 10B, pointers 8 bytes
    - (8+ (10+8)*M-1) = 4096
    - M = 228; 2.7 billion nodes in 4 levels
  - One block read per node visited